Checkpoint 2

Our Software Development Lifecycle (SDLC) for the Campus Navigation App

Justification of the Chosen Approach

We will adopt a hybrid approach, combining **incremental** and **adaptive** elements. This choice is driven by our analysis of uncertainty drivers and feedback timing principles:

- **Problem uncertainty:** We do not yet know with certainty whether users (freshmen and international students) will truly value the mapping of secondary services (such as water dispensers) as much as classroom locations. Gathering evidence through actual user behavior will help us reduce this risk.
- **Solution uncertainty:** We need to test whether our application's design and the database currently under development correctly solve the navigation need. Technical experiments and prototypes will provide the necessary evidence to validate our solution.
- **Feedback timing:** Useful feedback must arrive before commitments become prohibitively expensive to revise. By initially releasing the map for a single building, we will obtain timely feedback from users, avoiding the need to reorganize or correct the entire database for all university campuses at a late stage—an operation that would incur high refactoring costs.

Lifecycle Stages (From Definition to Operation)

Our SDLC connects planning, building, checking, and operations in a continuous loop, allowing the team to work and learn simultaneously:

- **Enter (Definition):** Features enter the backlog based on core user needs. Work is prioritized by clearly defining who has the authority to authorize scope changes (e.g., adding the mapping of new service categories), thereby avoiding silent scope reductions.
- **Build (Construction):** The team designs and implements the solution incrementally. Quality checking will occur by physically testing on campus the accuracy of the coordinates and data entered into the database before approving the work.
- **Release (Deployment and Operation):** We will deploy the candidate solution in its operating environment by releasing one usable capability at a time (e.g., the first increment will allow users to locate classrooms; the second will add services like microwaves).
- **Learn (Observation and Adaptation):** We will monitor real conditions and actual app usage. Following an adaptive approach, the team will respond to new evidence by evaluating whether the data justifies revising the current plan (e.g., stopping the mapping of water dispensers if they are never searched for). If justified, the plan will be updated; otherwise, the current plan will be kept.

Tracing an Actual Work Item: "Map Microwaves in the Scientific Campus"

To see how our system operates, here is the path of a single work item:

- **Waiting State (Stock):** The idea sits in our intake list until a team member has capacity.

- **Active Work:** A developer physically locates the microwaves, adds their GPS coordinates to our database, and updates the UI to include a "Microwave" filter.
- **Handoff & Waiting State (Queue):** The developer finishes the entry and transfers the work. Because a *handoff* requires transferring both the work and its context (to avoid questions and rework), they provide clear notes. The item then enters a *review queue*, waiting for reviewer attention and capacity.
- **Active Verification:** A second team member verifies the data (physically or via official campus maps) and approves the change.
- **Waiting State (Ready for Release):** The approved microwave locations sit in a staging state until the next update is pushed to the live app.
- **Completion:** The data is deployed. Freshmen can now actively search for microwaves on the live app.

Asynchronous Work & Required Conditions

To avoid bottlenecks and constant synchronization meetings, our team operates asynchronously. For example, one person can map Building A on Tuesday morning, while another codes the search bar interface on Wednesday night.

To support this flow, the following conditions must be in place:

- **Clear "Definition of Done":** Everyone must know exactly what constitutes a complete task (e.g., "A map point must include latitude, longitude, category, and a photo") to minimize rework.
- **Decoupled Architecture:** We need to be able to update database pins without breaking or waiting for the app's user interface code.
- **Centralized Tracking:** A shared digital board where everyone can see the current state of a task (Active, Waiting for Review, Done) to avoid duplicating work and make handoffs visible.
- **Delegated Authority:** Team members must have the autonomy to approve minor verification checks independently, rather than waiting for a full team meeting to authorize every single map pin.

Asynchronous Work & Required Conditions To avoid bottlenecks and the need for constant synchronous meetings, our team operates asynchronously, allowing people to contribute at different times. To sustain this, the following conditions must be in place:

- **Shared Records & Work Boundaries:** We rely on centralized tracking and clear task boundaries to preserve necessary context across handoffs without requiring a live conversation.
- **Decision Owners & Response Windows:** Every asynchronous decision has a named owner and a defined *response window*. This makes waiting time visible and prevents tasks from aging indefinitely in an approval queue.
- **Maintaining Slack:** Instead of aiming for 100% *utilization* (which turns normal task variation into long waiting queues), we maintain *slack*—capacity kept available to absorb unexpected difficulties or to help unblock reviews.

- **Paired Measures:** We measure our progress carefully. To ensure speed does not displace quality, we use *paired measures*: for example, we pair a flow measure (how fast we map a campus) with a quality guardrail (how many coordinate defects escape to the live app) to prevent hidden harm.